

Ablation Protocol, Version 2: Is the `r2py` Framework Necessary, and Does Each of Its Steps Earn Its Place?

A Three-Level Design

July 19, 2026

1 Why This Version Exists

The first version of this protocol (`ablation_protocol_20260719.tex`, retained unchanged) tested a single component—the pre-generated conversion guides—at a single granularity: it held the *entire* framework fixed, began from an already-validated package, and toggled the guides one function at a time. That design has a clean virtue (tight causal isolation of the guide effect) but a decisive blind spot. By presupposing a validated package as its substrate, it silently assumes that structural analysis, topological conversion ordering, whole-package assembly, and—for `rpart`—the five-phase foreign-function-interface (FFI) prologue have all already succeeded. It therefore cannot speak to the value of any of those global steps, and it quietly treats the framework’s most substantial contribution as free background.

It also tests only half of what the project actually claims. The methodology makes two distinct assertions: (a) that the framework as a whole is *necessary*—an unguided agent cannot reliably convert a nontrivial R package on its own; and (b) that each individual step, including the guides, *earns its place*. Version 1 addressed only (b), and only for one step. Claim (a) is the stronger and more important one, especially for a large `.Call()`-based package such as `rpart`, where without the framework—including the five prologue phases—no conversion is possible at all.

This version restructures the study into three nested levels, ordered from the most consequential claim to the most finely isolated. The Version-1 experiment survives intact as the innermost level.

	Question	Granularity	Claim served
L1	Is the framework necessary at all?	Whole package	(a) Necessity / feasibility
–	Is the prologue a hard prerequisite?	Whole package (build)	(a), for <code>.Call()</code> packages
L2	Which global steps matter, and how much?	Whole package	(b), for the <i>architectural</i> steps
L3	Do the guides earn their place?	Single function	(b), for the guides (Version 1)

The three levels answer three different reviewer questions. L1 answers “isn’t this just calling an LLM?” L2 answers “are all these phases needed, or is this over-engineered?” L3 answers “can

you prove the guides specifically do anything?” A study that runs only L3, as Version 1 did, inadvertently *undersells* the paper by implying the contribution is merely better prompting.

2 Level 1: Is the Framework Necessary?

2.1 Hypothesis

HA (Feasibility). An unstructured agent, given the same source, the same model, and a generous budget but none of the framework’s phased decomposition, dependency ordering, guide catalogue, or prologue scaffolding, fails to produce an installable, numerically faithful Python port; and the gap between the framework and the unstructured baseline widens with the target package’s structural difficulty.

The second clause matters as much as the first. `KernSmooth` is the easy quadrant—a single R file, plain-array `.Fortran()` interface, no prologue required—so an unstructured baseline may get part of the way. For `rpart`, whose `.Call()` backend cannot even be built without the reconstructed headers, the prediction is a hard failure. If borne out, this is not “the framework improves quality by $X\%$ ” but “the framework is the difference between a working package and nothing,” and the effect scaling with difficulty directly corroborates the paper’s central thesis (a fixed core plus a prologue whose cost grows with the FFI).

2.2 Conditions

Condition	What the agent is given	Role
F (framework)	The published pipeline: all phases, in order, with their skills and sub-agents.	Treatment. The delivered result already instantiates this.
U (unstructured)	The complete R package source and a single good-faith instruction to convert it to an installable Python package, with a generous iteration budget and the freedom to build, test, and self-correct; <i>no</i> phase decomposition, ordering, guide catalogue, or prologue scaffolding.	Control. Isolates the framework itself as the manipulated factor.

The baseline must be constructed in good faith. The single most likely reviewer objection to L1 is that the unstructured arm is a straw man. Three commitments forestall it. First, U receives a *capable* general-purpose agent, not a single-shot prompt: it may run tools, compile, read errors, and iterate, so that any failure is a failure of the approach and not of effort. Second, U’s instruction, its tool access, and its iteration and token budget are written down and committed to the repository *before* any run, exactly as the treatment prompts are. Third, the budget granted to U is at least as large as the total budget the framework consumed on the same package, so U cannot be said to have been starved.

2.3 Outcome: a feasibility ladder, not a pass rate

At the whole-package level the outcome is not a single number but an ordered set of milestones, each strictly harder than the last. Report the highest rung each run reaches.

1. Produces a package that **installs** (`pip install` succeeds).
2. For compiled backends, the native code **builds** (links without unresolved symbols).
3. The public API is **importable and runs** without exceptions on a canonical example (report the fraction of public functions that do).
4. Output is **numerically faithful** to R within the project’s declared tolerance (report the fraction of the reference test suite passed).

Milestones 1–3 are mechanical to score. Milestone 4 reuses the existing R-benchmarked suites, so it too is automated. The prediction is that F reaches milestone 4 on both packages (it already has), while U stalls at a low rung on `rpart`—plausibly failing milestone 2—and reaches a higher but still incomplete rung on `KernSmooth`.

2.4 Replication

Because HA is a threshold claim rather than a claim about a mean, it does not require the heavy replication of a continuous effect. A small number of independent U runs per package (target 3–5), each to its committed budget, suffices: if `rpart`’s backend fails to build across every U attempt, the point is made, and additional runs add little. Each U run’s transcript is archived so the failure mode is auditable rather than asserted.

3 The Prologue as a Hard Gate

The five prologue phases are not a quality knob that can be turned down by degrees; they are a binary prerequisite, and are best demonstrated by construction rather than by a graded experiment.

Demonstration. Attempt to compile `rpart`’s original, unmodified C sources against the standard toolchain with the reconstructed headers *absent*. The result is a set of link (or compile) errors against the R C API symbols the package references—the 51 identifiers the prologue was built to supply. This is a reproducible, one-command demonstration, not a statistical estimate, and it can be shown directly in the supplementary material as a captured build log.

Why there is no cheap escape. An unstructured path around the prologue has only two exits, and both are self-defeating. Embedding a live R runtime to satisfy the symbols reintroduces exactly the dependency the port exists to remove, so the result is not a native package. Hand-writing standalone replacements for the referenced symbols *is* the prologue—it is the framework’s own method, arrived at by a different name. The prologue is therefore not one option among several but the enabling mechanism itself, and the paper should state it as such: for a `.Call()`-based package, the reconstructed C API is a precondition for the existence of a backend, not an optimisation applied to one.

4 Level 2: Which Global Steps Matter?

4.1 Hypothesis

HB (Architectural steps). Removing an individual *global* step while retaining the rest of the framework degrades whole-package correctness or sharply increases the rework required to

reach it.

L2 sits between the all-or-nothing comparison of L1 and the single-function isolation of L3: the framework is otherwise intact, but one architectural step is disabled, and conversion is still performed over the whole package.

4.2 Conditions

Each condition below is the full framework minus exactly one global step.

Step removed	How it is disabled	Predicted failure mode
Topological ordering	Convert functions in an order independent of the dependency graph (e.g. alphabetical) rather than callees before callers.	A caller is converted before its callees exist, so their Python signatures must be guessed; more broken call expressions and more downstream rework.
Structural analysis	Withhold the per-file dependency JSON, so each agent must infer a function’s dependencies and FFI crossings unaided.	Missed or misclassified dependencies; inconsistent handling of the compiled-code boundary.
Guide catalogue	Convert the whole package with the guide-injection step removed (the L3 manipulation, lifted to whole-package scope).	Semantic drift between independently converted functions; the failure class L3 measures in isolation, observed here in aggregate.

The prologue is deliberately *absent* from this table: it is not degree-ablatable and is handled as a gate (previous section). Each L2 condition otherwise differs from the full framework by exactly one manipulable step, and the disabling procedure is committed in advance.

4.3 Outcome

Whole-package correctness, measured as the fraction of the reference suite passed after conversion, together with a count of correction cycles required to reach a build-and-import-clean state (a proxy for rework). Because L2 runs are whole-package and therefore costly, replication is modest (target 3 per condition per package); L2 is powered to detect gross degradation and rework inflation, not small effects, which are the province of L3.

5 Level 3: Do the Guides Earn Their Place? (Version 1, Nested)

This level is the Version-1 experiment, retained essentially unchanged and repositioned as the finest-grained arm. Its purpose is the tightest available causal identification of the *guide* effect specifically, with per-trial isolation that the whole-package levels cannot provide. It is summarised here; the original document holds the full detail.

5.1 Conditions

Three conditions differing only in the context supplied to the `@convert-r-function-to-python` sub-agent: **G** (full guide set), **N** (guide-injection block deleted, nothing else changed), and **P** (a generic, construct-agnostic instruction block of comparable token length, controlling for context volume). N is produced from G by deletion only; both, and the placebo block, are committed before any run.

5.2 Substitution harness

The unit is a (function, condition, replicate) triple. One function is converted, spliced into the otherwise-validated released package in place of its implementation, and evaluated only against the tests that exercise it (that subset fixed in advance by mutation). This isolates the per-function effect and prevents an early bad conversion from cascading. Functions with an empty exercising subset are excluded and the exclusion reported.

5.3 Outcomes

Correctness (HC). Binary per trial: do all exercising tests pass? Graded automatically against the R-benchmarked oracle.

Consistency (HD). Idiom dispersion D_k : the number of distinct normalised Python idioms used to translate construct k across its call sites within one run, over constructs with at least three sites. The normaliser is frozen before data collection. $D_k = 1$ means the construct was translated identically everywhere. To our knowledge this metric is novel and worth naming as a small methodological contribution.

Error character (HE, exploratory). Failures classified mechanically as silent (value assertion fails, no exception), loud (an exception), or structural (shape or type mismatch). The prediction is that removing guides shifts failures toward the *silent* category the methodology exists to prevent.

5.4 Replication

A 30-invocation pilot (5 functions $\times$ G/N $\times$ 3) estimates between-replicate variance; absent that, plan on 10 replicates per cell. Trials run in randomised order over (function, condition, replicate); the model identifier and sampling parameters are pinned and logged.

6 Contamination: The Principal Threat (All Levels)

Both released packages are public on GitHub and PyPI, and the upstream R packages have been public for decades, so a model may reproduce a known-correct translation from memory rather than derive it. This threat operates across all three levels and is handled the same way at each.

It biases toward the null. Memory helps the *weaker* arm in every comparison—the unstructured baseline at L1, the step-removed condition at L2, the no-guide condition at L3. A measured framework or guide effect is therefore a lower bound, and a significant positive result is conservative. This argument belongs in the paper: it converts the threat into a strengthening qualifier.

Memorisation probe. Prompt the agent for a function’s Python translation with *no* R source supplied, and score similarity against the released implementation. High similarity is direct evidence of memorisation and quantifies the contamination floor. Run across the function sample and report the distribution.

Held-out package (strongest control). Run a reduced version of the study on a package this project has never converted or published—**acepack** or **logspline**, both named as generality candidates in the May 11 addendum. A held-out arm removes the contamination objection outright

for that arm and simultaneously supplies the third-package generality data point the venue analysis contemplated. This is the single highest-value optional addition to the study; if run, prioritise it at L1 (feasibility), where the necessity claim is most valuable and least confounded.

7 Randomisation, Blinding, and Pre-Registration

Randomised order. Within each level, trials execute in randomised order over their design factors, not grouped by condition, so any drift in model behaviour distributes across conditions rather than confounding with one.

Blind analysis. Outcomes are recorded against opaque condition labels; the mapping is not revealed until the analysis script is frozen. With automated grading this is cheap to enforce honestly.

Pre-registration. Before the first non-pilot trial, commit and tag: the treatment definitions for all three levels; the U instruction and its budget; the L2 disabling procedures; the L3 prompt variants and their diff; the idiom normaliser; the per-function exercising-test map; the feasibility-ladder scoring rubric; and the analysis script. Selective venues respond well to a timestamped pre-registration, and it costs essentially nothing here.

8 Analysis Plan

Fixed in advance; deviations reported as such.

L1 (feasibility). Report the highest ladder rung reached by each run, by condition and package, as a small table. The claim is qualitative and threshold-shaped; no significance test is required, and the archived build logs and transcripts are the evidence. State the difficulty-scaling prediction (`rpart gap > KernSmooth gap`) in advance.

L2 (architectural steps). Per condition, report whole-package pass rate and rework-cycle count against the full framework, with the modest replication treated descriptively or with a rank test; L2 targets gross effects.

L3 (guides). As Version 1: a mixed-effects logistic regression of the binary pass outcome on condition with a per-function random intercept, reported as a marginal difference in pass rate with a confidence interval, plus a paired Wilcoxon signed-rank robustness check; idiom dispersion compared by paired Wilcoxon across constructs; failure taxonomy as a contingency table.

Reporting discipline. Report effect sizes with confidence intervals, not p -values alone. Report all pre-registered outcomes, including null ones. A null at any level is publishable and informative: a null L1 would be a striking finding about model capability; a null L3 would narrow the guides' contribution to consistency and auditability rather than raw correctness. Committing to report nulls in advance is what makes the positive results credible.

9 Threats to Validity

Straw-man baseline (L1). The gravest risk to the necessity claim. Mitigated by the good-faith construction of U: capable agent, iteration allowed, budget at least matching the framework’s, all pre-committed.

“Impossible” overreach. The paper must not assert metaphysical impossibility. The defensible claim is operational: within a good-faith, well-resourced unstructured effort, no installable and numerically faithful package resulted, and the prologue’s two escape hatches each defeat the port’s purpose. State it that way.

Contamination. Biases toward the null at every level; quantified by the memorisation probe; eliminated in the held-out arm.

Oracle incompleteness. The suites cannot detect a silent error on an unexercised path, so measured pass rates upper-bound true correctness in every arm. This inflates all arms and should not bias comparisons, but it bounds absolute claims and must be stated.

Cost of the whole-package levels. L1 and L2 are expensive and modestly replicated. They are powered for threshold and gross effects; fine effects are deliberately left to L3, whose isolation and replication are built for them. The three levels are complementary by construction.

Single model family. Results hold for one model. The tool-agnostic claim is not established by this study; state the limitation, and if cheap, repeat a reduced arm on a second model family.

10 Execution Sequence

1. Capture the prologue-gate build log (**rpart** C sources without the reconstructed headers). One command; do first.
2. Write and commit all pre-registered artifacts: the U instruction and budget, the L2 disabling procedures, the L3 prompts and normaliser, the exercising-test map, and the feasibility rubric. Tag.
3. Run the memorisation probe; record the contamination floor.
4. Run the L3 pilot (30 invocations); set replication.
5. Pre-register the full protocol and analysis script. Tag.
6. Execute L1 (feasibility) on both packages, archiving every transcript.
7. Execute L3 (guides) on the **KernSmooth** primary tier, then the **rpart** confirmatory tier.
8. Execute L2 (architectural steps) on both packages.
9. Optional, recommended: the held-out arm, prioritised at L1.
10. Unblind, run the frozen analysis, report every pre-registered outcome.

Steps 1–5 are mechanical engineering against existing infrastructure. Steps 6–9 are compute time. L1 and the prologue gate are the additions that make the study speak to the framework’s necessity rather than only to the guides.

11 What This Contributes to the Manuscript

- **The necessity result (L1) is the headline.** It answers the most dangerous reviewer question—“isn’t this just an LLM call?”—with evidence rather than assertion, and its difficulty-scaling form corroborates the core-plus-prologue thesis directly. The abstract’s ablation sentence should lead with this (the framework enables what an unstructured agent cannot), demoting the guide percentage to a component result.
- **The prologue gate** converts “.Call() is harder” into “.Call() is impossible without a reconstructed C API,” shown by a build log rather than argued.
- **The architectural ablations (L2)** show that ordering and structural analysis are load-bearing, not decorative—defending the phase count against an over-engineering charge.
- **The guide ablation (L3)** supplies the tightest causal estimate of the guides’ effect, plus the idiom-dispersion metric and, if it holds, the silent-vs-loud failure shift that distinguishes this work from general code translation.
- **A three-level, pre-registered design** is unusual in this literature and is cheap credibility at a selective venue.

Downstream, this changes Results §5 and the abstract: the ablation is no longer a single guide-effect number but a graded argument running from “the framework is necessary” (L1), through “the prologue is a precondition” (gate), to “each architectural step is load-bearing” (L2) and “the guides specifically earn their place” (L3). Those edits follow once results exist.